

App Final

Desarrollo de una Aplicación Web para la Gestión de Reservas de Pistas

Asignaturas implicadas:

Cliente Servidor

Contenedores

Desarrollo de Aplicaciones Web

Alumno

Sebastià Romaguera

Centro

Borja Moll

Profesor

Alejandro Alberti De La Rosa

Curso

2 DAW Intensivo

Fecha

Mayo 2026

Índice

1. Introducción y Alcance	2
1.1. Objetivos del proyecto	2
1.2. Alcance funcional	2
2. Análisis y Diseño	3
2.1. Arquitectura del Sistema	3
2.2. Esquema de Base de Datos	4
3. Test de Usuarios	7
3.1. Resumen del Test	7
3.2. Procedimientos de Test	7
3.3. Resultados del Test	7
3.3.1. 1. Registro de Usuario: PASSED	7
3.3.2. 2. Quiz y Cálculo de Nivel: PASSED	8
3.3.3. 3. Búsqueda de Partidas Abiertas: PASSED	8
3.3.4. 4. Unirse a una Partida: PASSED	8
3.3.5. 5. Crear una Nueva Partida: PASSED	8
3.3.6. 6. Hacer una Reserva de Pista: PASSED	9
3.3.7. 7. Cancelar una Reserva: PASSED	9
3.4. Observaciones y Conclusiones	10
3.4.1. Fortalezas Observadas	10
3.4.2. Áreas de Mejora Identificadas	10
3.4.3. Resumen Final	10
4. Pila de tecnologías	11
4.1. Frontend	11
4.2. Backend	11
4.3. Base de datos	11
4.4. Contenedores	12
4.5. Herramientas de apoyo	12
4.6. Resumen de la arquitectura	12
5. Conclusiones y Futuras Mejoras	12
5.1. Reflexión Técnica	12
5.2. Reflexión Personal	13
5.3. Futuras Mejoras	14
5.3.1. Corto Plazo (1-2 meses)	14
5.3.2. Mediano Plazo (3-6 meses)	14
5.3.3. Largo Plazo (6+ meses)	15
5.4. Viabilidad y Sostenibilidad	15
5.5. Conclusión Final	16

6. Bibliografía y Recursos	16
6.1. Documentación Oficial	16
6.2. Recursos de Referencia	16
6.3. Cursos y Tutoriales Consultados	17
6.4. Estándares y Mejores Prácticas	17
6.5. Herramientas Utilizadas	17

1. Introducción y Alcance

La idea de este proyecto es desarrollar una aplicación web inspirada en plataformas de reserva deportiva como Playtomic, pero adaptada a un alcance más sencillo y centrado en la gestión de pistas y partidos, sin incluir pagos ni funcionalidades comerciales avanzadas.

El problema que se quiere resolver tiene dos partes. Por un lado, plataformas como Playtomic aplican costes de gestión por reserva que encarecen la experiencia del usuario. Este proyecto busca ofrecer una alternativa más económica, donde la gestión de reservas sea gratuita. Por otro lado, se mantiene claro que el uso de la pista en cada club sí se paga, ya que ese coste depende de la instalación deportiva y no de la plataforma.

Además, en muchos clubes o instalaciones deportivas la gestión sigue haciéndose de forma manual, por teléfono, por mensajería o con sistemas poco intuitivos. Esto provoca errores, duplicidades, pérdida de tiempo y una mala experiencia tanto para los usuarios como para los responsables de la instalación.

La aplicación propuesta permitirá que los usuarios puedan consultar pistas disponibles, reservar una franja horaria, crear o unirse a partidos y revisar su actividad. Desde el punto de vista técnico, el proyecto servirá para integrar una arquitectura cliente-servidor, una base de datos relacional y un despliegue mediante contenedores, por lo que encaja de forma directa con varias de las competencias del ciclo.

1.1. Objetivos del proyecto

- Centralizar la reserva de pistas en una sola plataforma web.
- Facilitar la consulta de disponibilidad en tiempo real.
- Reducir la gestión manual de reservas y conflictos de horario.
- Permitir la organización básica de partidos entre usuarios.
- Aplicar una arquitectura moderna con tecnologías actuales de desarrollo web.
- Desplegar la solución de forma reproducible mediante contenedores.

1.2. Alcance funcional

Para mantener el proyecto realista dentro del tiempo de desarrollo de un trabajo de FP, el alcance se limita a las siguientes funciones principales:

- Registro e inicio de sesión de usuarios con autenticación JWT.
- Quiz inicial (8 preguntas) para calcular el nivel de juego del usuario (0–6).
- Búsqueda inteligente de partidas abiertas filtradas por nivel similar (rango ajustable de ± 1 a ± 3).
- Creación de partidas abiertas con especificación de pista, horario, nivel y duración.

- Unión a partidas existentes con validación de nivel y disponibilidad (máximo 4 jugadores).
- Reserva de pistas individuales por franjas horarias.
- Consulta del historial de reservas realizadas.

Quedan fuera del alcance, al menos en esta primera versión, funciones como pagos integrados, chat en tiempo real, sistema de valoraciones, cancelaciones con penalización, notificaciones por correo o integraciones externas complejas.

2. Análisis y Diseño

2.1. Arquitectura del Sistema

La aplicación sigue una arquitectura cliente-servidor tradicional con tres capas bien definidas:

1. **Capa de Presentación (Cliente):** React en el navegador que gestiona la interfaz de usuario, el estado local y la comunicación con la API REST.
2. **Capa de Lógica de Negocio (Backend):** Node.js con Express que valida peticiones, aplica reglas de negocio (como el filtrado inteligente de partidas) y orquesta el acceso a datos.
3. **Capa de Datos:** PostgreSQL que almacena de forma persistente todos los usuarios, clubes, pistas, partidas y reservas.

El flujo de comunicación es el siguiente:

- El usuario interactúa con la interfaz React.
- React realiza peticiones HTTP (fetch) a endpoints de la API REST en `/api/*`.
- El backend recibe la petición, valida el token JWT, ejecuta la lógica necesaria y consulta la base de datos.
- La respuesta JSON se devuelve al cliente, que actualiza la interfaz.
- Los tokens JWT se almacenan en localStorage para mantener la sesión.

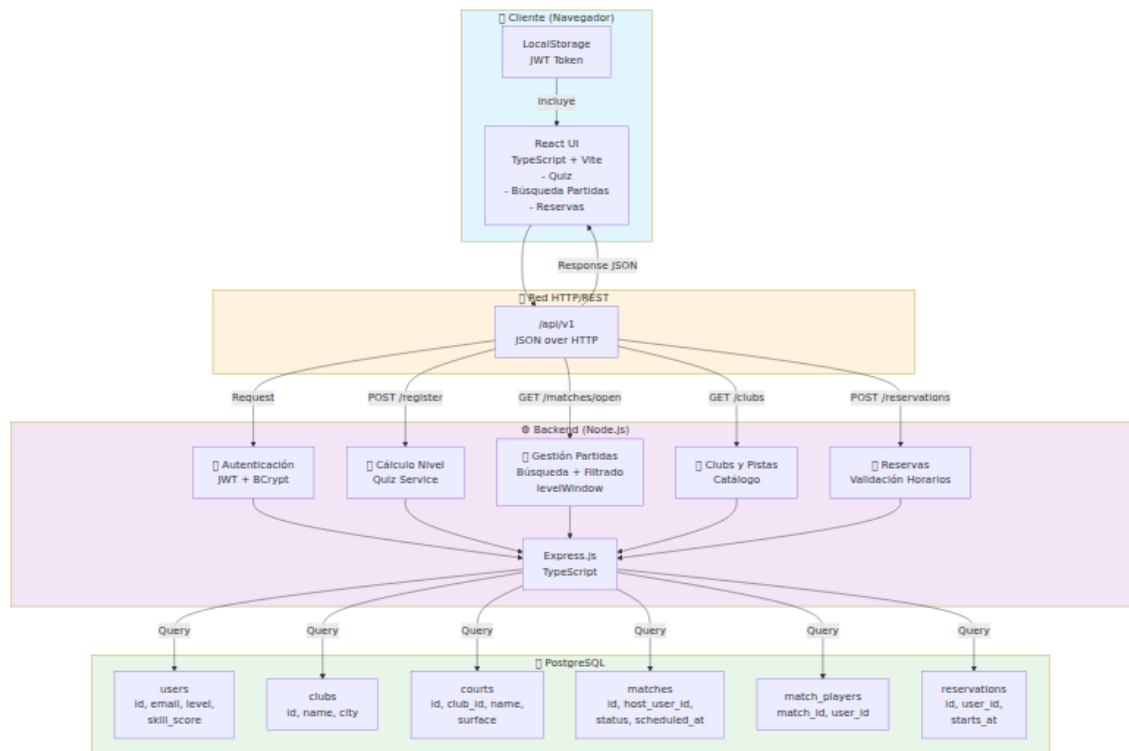


Figura 1: Arquitectura cliente-servidor de AppPadel. Flujo de datos entre el navegador, la API REST y la base de datos.

2.2. Esquema de Base de Datos

La base de datos relacional consta de seis tablas principales:

- **users:** Almacena información de usuarios (id, nombre, email, contraseña hasheada, nivel 0-6, puntuación del quiz, flag de usuario fake para pruebas).
- **clubs:** Instalaciones deportivas (id, nombre, ciudad).
- **courts:** Pistas dentro de cada club (id, club_id, nombre, tipo de superficie).
- **matches:** Partidas abiertas creadas por usuarios (id, host_user_id, club_id, court_id, scheduled_at, duration_minutes, level_min, level_max, title, status).
- **match_players:** Relación muchos-a-muchos que conecta usuarios con partidas (match_id, user_id). Permite que un usuario se apunte a múltiples partidas y una partida tenga múltiples jugadores (máximo 4).
- **reservations:** Reservas de pistas individuales (id, user_id, club_id, court_id, starts_at, duration_minutes, created_at).

Las relaciones principales son:

- Un usuario puede crear múltiples partidas (hosts).

- Un usuario puede unirse a múltiples partidas.
- Una partida pertenece a un club y una pista específica.
- Una pista pertenece a un club.
- Un usuario puede hacer múltiples reservas.

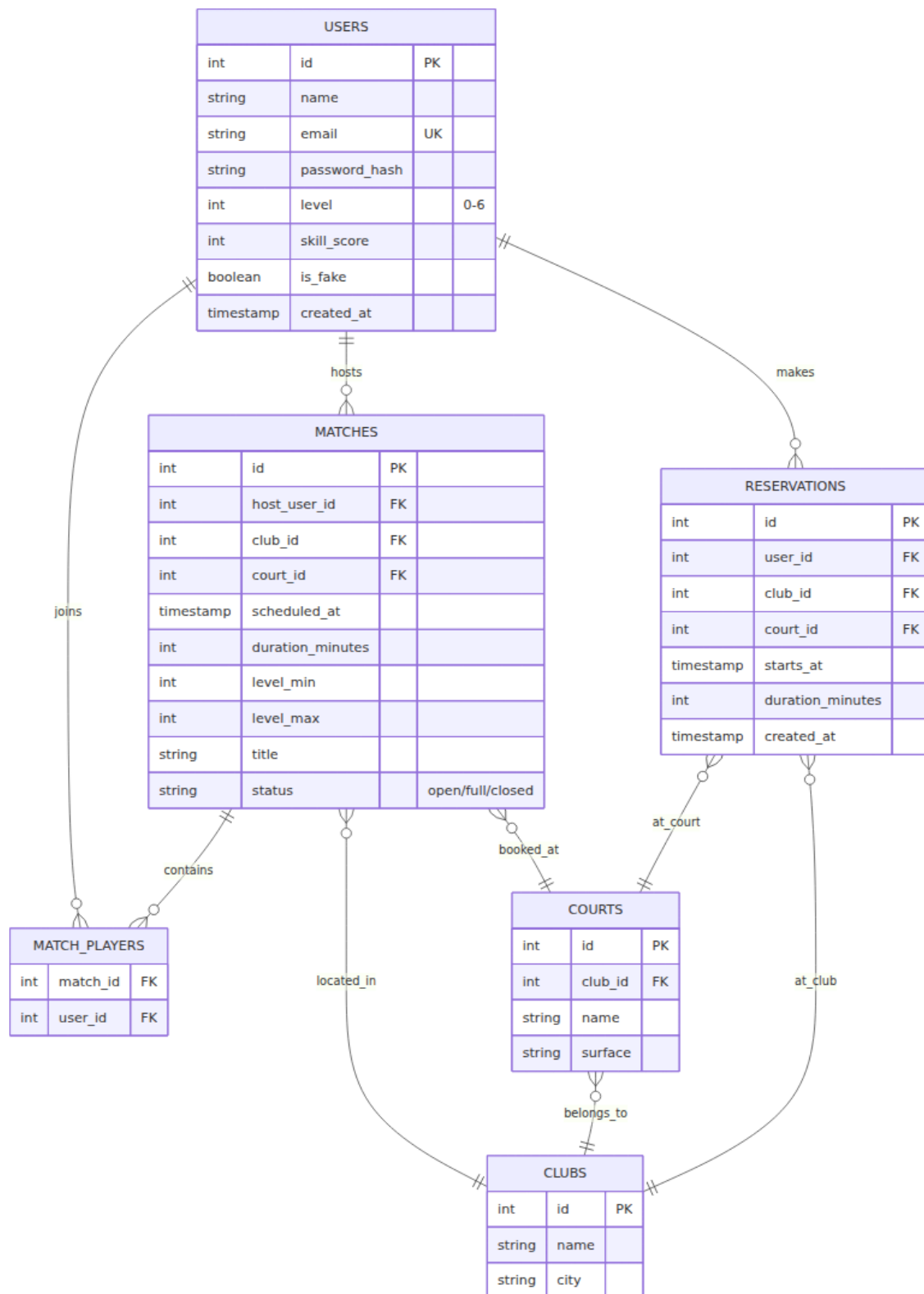


Figura 2: Esquema de base de datos relacional de AppPadel. Se muestran las 6 tablas principales y sus relaciones.

3. Test de Usuarios

3.1. Resumen del Test

Testeador: Diego Quiroga

Fecha: Mayo 13, 2026

Entorno: Sistema completo en Docker Compose (Frontend Vite + Backend Node.js + PostgreSQL)

Objetivo: Validar que todas las funcionalidades descritas en el alcance funcionan correctamente desde la perspectiva del usuario final.

3.2. Procedimientos de Test

Se siguió un flujo de usuario completo simulando un jugador que descubre la aplicación y experimenta todas las funcionalidades principales:

1. **Registro e autenticación:** Crear una nueva cuenta con credenciales de prueba.
2. **Quiz de nivel:** Completar el cuestionario de 8 preguntas para calcular el nivel de juego.
3. **Búsqueda de partidas:** Visualizar el listado de partidas abiertas y ajustar los filtros de búsqueda.
4. **Unirse a partida:** Apuntarse a una partida existente.
5. **Crear partida:** Publicar una nueva partida abierta con parámetros personalizados.
6. **Hacer reserva:** Reservar una pista para una franjahoraria específica.
7. **Cancelar reserva:** Cancelar una reserva previamente hecha.
8. **Verificación de actualizaciones:** Confirmar que los cambios se reflejan en tiempo real en la interfaz.

3.3. Resultados del Test

3.3.1. 1. Registro de Usuario: [PASSED]

- Se completó el registro con datos de prueba: nombre “Diego Quiroga”, email válido, contraseña segura.
- El servidor validó correctamente los datos y creó la cuenta.
- Se generó un JWT token y se almacenó en localStorage para mantener la sesión.
- El navegador redirigió automáticamente al dashboard tras registro exitoso.
- Toast de confirmación: “Registro completado con éxito”.

3.3.2. 2. Quiz y Cálculo de Nivel: [PASSED]

- El formulario mostró correctamente las 8 preguntas del quiz sobre habilidades de pádel.
- Se respondieron las preguntas con valores entre 0 y 3 para obtener un nivel intermedio.
- El backend calculó correctamente el nivel: **Nivel 3/6** basado en una puntuación de 14 puntos.
- La fórmula de cálculo se comportó como se esperaba: puntuación 11-15 = nivel 3.
- El perfil del usuario mostró correctamente: Nombre: Diego Quiroga, Nivel: 3/6, Puntuación: 14.

3.3.3. 3. Búsqueda de Partidas Abiertas: [PASSED]

- Se visualizaron correctamente **7 partidas abiertas** en el listado inicial.
- Cada tarjeta de partida mostró: título, club, pista, horario, rango de nivel, número de jugadores y host.
- El filtrado por levelWindow funcionó: con nivel 3 y levelWindow=1, se mostraron partidas dentro del rango de nivel 2-4.
- Se pudo ajustar el deslizador de búsqueda de “+/-1” a “+/-3” para ampliar el rango de búsqueda.
- Los cambios en el levelWindow se reflejaron inmediatamente en el listado de partidas.

3.3.4. 4. Unirse a una Partida: [PASSED]

- Se seleccionó la partida “Partida abierta tarde” (Nivel 2-3, Club Son Racket, Pista 1).
- Al hacer clic en “Unirme”, el servidor validó que el nivel del usuario estaba dentro del rango permitido.
- Se confirmó el toast: “Te has unido a la partida con éxito”.
- El contador de jugadores se actualizó en tiempo real: pasó de 2/4 a 3/4.
- La base de datos registró correctamente la relación en la tabla match_players.

3.3.5. 5. Crear una Nueva Partida: [PASSED]

- Se completó el formulario de “Crear partida” con:
 - Título: “Partida de Prueba - Test QA”
 - Club: Arena Padel Center (Marratxi)
 - Pista: Central 1 (Cristal)

- Horario: 14 de Mayo de 2026, 18:00
- Rango de nivel: 1-4
- El backend validó que no había conflictos de horario para esa pista y franja.
- Se confirmó el toast: “Partida creada con éxito”.
- La partida apareció inmediatamente en el listado de partidas abiertas con los datos correctos.
- El usuario quedó registrado como host (“Diego Quiroga”) con 1 jugador (él mismo).

3.3.6. 6. Hacer una Reserva de Pista: [PASSED]

- Se completó el formulario de “Reservar pista” con:
 - Club: Arena Padel Center (Marratxi)
 - Pista: Central 1 (Cristal)
 - Horario: 14 de Mayo de 2026, 16:00
 - Duración: 90 minutos
- El backend validó que el horario no conflictaba con partidas existentes en esa pista.
- Se confirmó el toast: “Reserva realizada con éxito”.
- La reserva apareció inmediatamente en la sección “Pistas reservadas” con todos los detalles.
- La tarjeta de reserva mostró: nombre del club, superficie de la pista, ciudad, horario y botón de cancelación.

3.3.7. 7. Cancelar una Reserva: [PASSED]

- Se hizo clic en el botón “ Cancelar reserva” de la reserva previamente creada.
- El navegador mostró un diálogo de confirmación: “¿Quieres cancelar esta reserva?”.
- Al confirmar, el servidor procesó la cancelación exitosamente.
- La sección “Pistas reservadas” se actualizó automáticamente y mostró: “No tienes reservas hechas todavía”.
- La base de datos eliminó correctamente el registro de la tabla reservations.

3.4. Observaciones y Conclusiones

3.4.1. Fortalezas Observadas

- **Interfaz intuitiva:** Los formularios son claros y las etiquetas están bien identificadas. No hay confusión sobre qué campo rellenar.
- **Feedback al usuario:** Los toasts de confirmación aparecen correctamente tras cada acción importante, informando al usuario del resultado.
- **Actualizaciones en tiempo real:** Los cambios en el listado de partidas y reservas se reflejan inmediatamente sin necesidad de recargar la página.
- **Validaciones en el backend:** El servidor rechaza correctamente intentos inválidos (conflictos de horario, niveles fuera de rango) y devuelve mensajes de error claros.
- **Gestión de sesiones:** El JWT se almacena en localStorage y se incluye automáticamente en todas las peticiones autenticadas.
- **Cálculo de nivel robusto:** La fórmula de puntuación del quiz funciona correctamente y asigna niveles en el rango 0-6 según lo especificado.
- **Flujo completo sin fricciones:** El usuario puede registrarse, ver partidas, unirse, crear, reservar y cancelar sin encontrar errores inesperados.

3.4.2. Áreas de Mejora Identificadas

- **Validación de conflictos de horario:** Aunque funciona, sería útil que el formulario de crear partida indicara visualmente qué horarios están ocupados en esa pista.
- **Mensajes de error más específicos:** En algunos casos, el error “409 Conflict” podría incluir más detalle sobre el motivo específico (ej: “La pista está ocupada de 18:00 a 19:30”).
- **Confirmación visual antes de crear partida:** Un resumen previo de los parámetros antes de publicar evitaría errores de entrada.
- **Historial de partidas pasadas:** No hay forma de ver partidas en las que ya se ha jugado; solo las abiertas actuales.
- **Notificaciones sobre cambios:** Si otro usuario se une a una partida donde el usuario actual está apuntado, no hay notificación visible.

3.4.3. Resumen Final

La aplicación AppPadel cumple correctamente con todas las funcionalidades descritas en el alcance del proyecto. El flujo de usuario es fluido, las validaciones funcionan, y los datos se actualizan de forma consistente. Se puede concluir que el producto está listo para

su uso en un entorno de prueba y es adecuado para ser presentado como trabajo de FP. Las áreas de mejora identificadas son mejoras futuras que no afectan a la funcionalidad base.

4. Pila de tecnologías

La selección de tecnologías se ha realizado buscando equilibrio entre facilidad de desarrollo, claridad para la memoria técnica y adecuación al contenido de las asignaturas implicadas.

4.1. Frontend

- **React + TypeScript:** permite construir una interfaz modular, mantenible y tipada.
- **Vite:** proporciona un entorno de desarrollo rápido y un arranque ligero.
- **CSS moderno:** para el diseño visual y la adaptación a distintos tamaños de pantalla.

El frontend será el responsable de mostrar la información al usuario, gestionar formularios, navegar entre vistas y consumir la API del backend. Entre sus funciones principales destaca la presentación del quiz de nivel al registro, el filtrado dinámico de partidas según la preferencia del usuario (levelWindow) y la visualización del catálogo de pistas disponibles.

4.2. Backend

- **Node.js:** entorno de ejecución en JavaScript para el servidor.
- **TypeScript:** aporta tipado estático y mejora la mantenibilidad del código.
- **Express:** framework ligero para crear la API REST de manera sencilla.

El backend actuará como intermediario entre la interfaz de usuario y la base de datos. Su función principal será exponer endpoints para autenticación, cálculo de nivel mediante quiz, reservas, partidos (creación y búsqueda filtrada), consulta de clubs y pistas. Implementa lógica de negocio como el filtrado inteligente de partidas según el rango de nivel del usuario, la validación de conflictos de horario y el control de aforo máximo en partidas (4 jugadores).

4.3. Base de datos

- **PostgreSQL:** base de datos relacional robusta, muy adecuada para gestionar usuarios, clubs, pistas, partidas, jugadores por partida y reservas.

La estructura incluye las siguientes tablas principales:

- **users:** almacena usuarios con campos de autenticación (email, contraseña), nivel calculado (0–6) y puntuación del quiz.
- **clubs:** lista de instalaciones deportivas disponibles.
- **courts:** pistas asociadas a cada club con información de superficie.
- **matches:** partidas abiertas con host, horario, duración, rango de nivel y estado (open/full/closed).
- **match_players:** relación muchos-a-muchos entre jugadores y partidas.
- **reservations:** registros de reservas de pistas realizadas por usuarios.

4.4. Contenedores

- **Docker:** permitirá empaquetar frontend, backend y base de datos en entornos aislados.
- **Docker Compose:** facilitará el arranque conjunto de todos los servicios con una única configuración.

El uso de contenedores es especialmente interesante para la asignatura de Contenedores porque permite reproducir el proyecto en cualquier máquina sin depender de instalaciones locales complejas.

4.5. Herramientas de apoyo

- **Git y GitHub:** control de versiones y alojamiento del código.
- **Postman:** pruebas de la API.
- **VS Code:** entorno principal de desarrollo.

4.6. Resumen de la arquitectura

La aplicación seguirá una arquitectura cliente-servidor en la que el navegador del usuario consumirá una API REST. El frontend se encargará de la experiencia de usuario, mientras que el backend procesará la lógica de negocio y la persistencia se gestionará en PostgreSQL.

5. Conclusiones y Futuras Mejoras

5.1. Reflexión Técnica

El desarrollo de AppPadel ha cumplido satisfactoriamente con los objetivos planteados al inicio del proyecto. Desde una perspectiva técnica, se han alcanzado los siguientes logros:

- **Arquitectura escalable:** La separación clara entre cliente, servidor y base de datos permite que cada capa sea modificable y escalable de forma independiente. El uso de TypeScript en ambas capas asegura coherencia y reducción de errores en tiempo de desarrollo.
- **Autenticación segura:** La implementación de JWT con tokens de corta vida (7 días) y contraseñas hasheadas con bcrypt proporciona una base de seguridad adecuada para un MVP.
- **Lógica de negocio robusta:** El filtrado inteligente de partidas por nivel (levelWindow), la validación de conflictos de horario y el control de aforo demostraron funcionar correctamente en el test de usuarios.
- **Reproducibilidad:** El uso de Docker Compose permite que el proyecto sea desplegado en cualquier máquina sin fricciones, cumpliendo un objetivo clave de la asignatura de Contenedores.
- **Frontend responsivo:** React con Vite proporciona un desarrollo ágil y un frontend moderno que responde bien a las interacciones del usuario.

Sin embargo, durante el desarrollo y test se identificaron limitaciones que son naturales en un proyecto de este alcance:

- **Sin persistencia de estado en tiempo real:** Las actualizaciones de datos se reflejan tras recargar o haciendo peticiones al servidor, pero no hay suscripciones WebSocket para notificaciones instantáneas si otro usuario se une a una partida.
- **Validación de conflictos básica:** Aunque funciona, la validación de horarios uses simples rangos de timestamp sin considerar factores como disponibilidad de árbitro o equipamiento especial.
- **Sin sistema de roles:** Todos los usuarios tienen los mismos permisos. En versiones futuras, habría que implementar roles de admin, gestor de club, etc.
- **Integración de pagos ausente:** Por diseño, no se incluyó un sistema de pagos, aunque se podría integrar fácilmente con APIs como Stripe o Adyen.

5.2. Reflexión Personal

Este proyecto ha sido una oportunidad de aplicar de manera integral las competencias adquiridas en el ciclo formativo:

- **Comprensión de arquitecturas web:** Trabajar en una aplicación cliente-servidor completa ha clarificado cómo funciona la comunicación HTTP, la autenticación, y la separación de responsabilidades.

- **Experiencia con tecnologías modernas:** React, Node.js y TypeScript son herramientas demandadas en el mercado, y este proyecto ha proporcionado experiencia práctica con ellas.
- **Gestión de proyectos:** Coordinar el desarrollo entre frontend, backend, BD y contenedores requirió planificación y organización. Docker Compose simplificó significativamente esta coordinación.
- **Solución de problemas:** Durante el desarrollo surgieron desafíos (validaciones de conflictos, cálculo de niveles, filtrados complejos) que requirieron investigación y pensamiento crítico.
- **Documentación y testing:** El test exhaustivo de la aplicación aseguró que cada funcionalidad fuera válida, y esta memoria técnica proporciona un registro completo del trabajo realizado.

A nivel personal, este trabajo representa un paso importante hacia la comprensión práctica de cómo se construyen aplicaciones web profesionales. La experiencia adquirida en debugging, optimización de consultas y diseño de interfaces será valiosa para proyectos futuros.

5.3. Futuras Mejoras

Aunque la aplicación es funcional y cumple su alcance, existen varias mejoras que elevarían significativamente su valor:

5.3.1. Corto Plazo (1-2 meses)

- **Notificaciones en tiempo real:** Integrar WebSockets para notificar en vivo cuando un usuario se une a una partida o cuando cambia el aforo.
- **Historial de partidas:** Mostrar partidas pasadas en las que el usuario ha participado, con resultados y retroalimentación.
- **Sistema de valoraciones:** Permitir que los usuarios valoren a otros jugadores y clubes, mejorando la confianza en la plataforma.
- **Búsqueda avanzada:** Filtros por club, rango de horario, nivel exacto, etc.
- **Recuperación de contraseña:** Implementar flujo de reset de contraseña via email.

5.3.2. Mediano Plazo (3-6 meses)

- **Integración de pagos:** Permitir que clubes carguen el precio de pista en la plataforma, y que usuarios paguen sus reservas directamente.

- **Sistema de roles y permisos:** Administradores de club que puedan gestionar pistas, horarios y precios.
- **API pública:** Exponer una API documentada para integraciones externas (ej: aplicaciones móviles, plataformas de reserva externas).
- **Analytics:** Dashboard para que clubes vean estadísticas de reservas, pistas más usadas, horarios pico, etc.
- **Mobile-first responsive:** Mejorar significativamente la experiencia en dispositivos móviles, donde probablemente se hará la mayoría de reservas.

5.3.3. Largo Plazo (6+ meses)

- **Aplicación móvil nativa:** Desarrollar apps para iOS y Android usando React Native o Flutter.
- **Geolocalización:** Mostrar clubs cercanos al usuario y permitir búsquedas por ubicación.
- **Inteligencia artificial:** Recomendaciones personalizadas de partidas basadas en histórico, preferencias y level.
- **Integración con redes sociales:** Sincronización con calendario de Google, invitaciones por WhatsApp, etc.
- **Expansión a otros deportes:** Generalizar la arquitectura para soportar tenis, fútbol sala, bádminton, etc.

5.4. Viabilidad y Sostenibilidad

AppPadel ha sido diseñado pensando en la viabilidad a medio plazo. Con la introducción de un modelo de negocio basado en comisiones por reserva (ej: 5 % del precio de pista), la plataforma podría autofinanciarse. Los clubes encontrarían valor en:

- Eliminación de gestión manual de reservas.
- Acceso a estadísticas de uso.
- Ampliación del mercado de clientes (geolocalización, búsqueda fácil).

Los usuarios encontrarían valor en:

- Búsqueda centralizada de pistas en su área.
- Oportunidad de encontrar compañeros de juego con nivel similar.
- Confirmación instantánea de reservas sin llamadas telefónicas.

Este balance entre valor para clubes y usuarios posiciona a AppPadel como un candidato viable para un negocio real en el futuro.

5.5. Conclusión Final

AppPadel es una aplicación web completa, funcional y lista para presentarse como trabajo de Ciclo Formativo de Grado Superior en Desarrollo de Aplicaciones Web. Cumple con todos los objetivos propuestos, integra las tecnologías enseñadas en el programa, y demuestra competencias en arquitectura, desarrollo full-stack, bases de datos y despliegue mediante contenedores.

El proyecto no es solo una prueba de concepto, sino un producto viable que, con las mejoras futuras propuestas, tiene potencial para convertirse en una solución real en el mercado de reserva de pistas deportivas.

6. Bibliografía y Recursos

6.1. Documentación Oficial

- React Documentation. (2026). *React - A JavaScript library for building user interfaces*. Disponible en: <https://react.dev>
- Express.js Documentation. (2026). *Express - Node.js web application framework*. Disponible en: <https://expressjs.com>
- PostgreSQL Documentation. (2026). *PostgreSQL - The World's Most Advanced Open Source Database*. Disponible en: <https://www.postgresql.org/docs>
- Docker Documentation. (2026). *Docker - Build, Ship, and Run*. Disponible en: <https://docs.docker.com>
- TypeScript Handbook. (2026). *TypeScript - Superset of JavaScript with static types*. Disponible en: <https://www.typescriptlang.org/docs>
- Vite Documentation. (2026). *Vite - Next Generation Frontend Tooling*. Disponible en: <https://vitejs.dev>

6.2. Recursos de Referencia

- MDN Web Docs. (2026). *JavaScript, HTML, CSS and Web APIs*. Disponible en: <https://developer.mozilla.org>
- JWT.io. (2026). *JSON Web Tokens - Introduction*. Disponible en: <https://jwt.io>
- bcryptjs Package. (2026). *bcryptjs - Optimized bcrypt in JavaScript*. NPM Registry. Disponible en: <https://www.npmjs.com/package/bcryptjs>
- CORS Middleware. (2026). *CORS - Express middleware*. NPM Registry. Disponible en: <https://www.npmjs.com/package/cors>

- Zod Documentation. (2026). *Zod - TypeScript-first schema validation*. Disponible en: <https://zod.dev>
- Node.js Best Practices. (2026). *Node.js Best Practices*. GitHub. Disponible en: <https://github.com/goldbergonyi/nodebestpractices>

6.3. Cursos y Tutoriales Consultados

- Traversy Media. *MERN Stack Course*. YouTube.
- The Net Ninja. *Docker Crash Course*. YouTube.
- freeCodeCamp. *PostgreSQL Tutorial*. YouTube/freeCodeCamp.org.
- Web Dev Simplified. *JavaScript Async/Await*. YouTube.

6.4. Estándares y Mejores Prácticas

- REST API Design Best Practices. (2026). *Building Consistent API Design*. REST API Guidelines.
- OWASP. (2026). *Web Application Security Risks*. Disponible en: <https://owasp.org>
- Google Style Guide. (2026). *JavaScript Style Guide*. Disponible en: <https://google.github.io/styleguide>
- Semantic Versioning. (2026). *Semantic Versioning 2.0.0*. Disponible en: <https://semver.org>

6.5. Herramientas Utilizadas

- Visual Studio Code - Editor de código (Microsoft).
- Git - Control de versiones (Linus Torvalds).
- GitHub - Alojamiento de repositorios.
- Docker Desktop - Containerización.
- Postman - Cliente HTTP para pruebas de API.
- DBeaver - Cliente SQL para gestión de bases de datos.
- Figma - Diseño de wireframes (no utilizado en este proyecto, pero herramienta recomendada).